Faster Widgets via Learned Sorting

A. Author and B. Writer

Abstract

We study widget sorting and propose a learned sorter that is faster than classical baselines. Accuracy rises from 0.76 to 0.87 on the benchmark suite.

1 Introduction

Sorting widgets is a core step in many pipelines and it is often the bottleneck. Existing methods ignore the structure of the data, e.g. nearly sorted inputs. We therefore ask whether a learned model can exploit that structure.

Our contributions are threefold and are summarised in the following sections of this paper.

2 Related Work

Classical comparison sorts were analysed by Knuth et al. in great detail. Learned index structures were introduced more recently and reduce lookup time. Hybrid approaches combine both ideas but require manual tuning of many parameters.

3 Proposed Method

Our method first computes an embedding of each widget with an extractive feature encoder that runs in linear time over the input stream. The embedding is then bucketed by a small neural predictor.

3.1 Model Architecture

The predictor has two hidden layers and a softmax output over buckets. Buckets are merged greedily until each one fits in cache. The final pass is an insertion sort over nearly ordered buckets.

Figure 1: Overview of the learned sorter. The encoder feeds the bucket predictor.

4 Experimental Setup

We evaluate on three synthetic datasets of one million widgets each. Baselines are quicksort, mergesort and a learned index. All runs use a single core and five random seeds.

5 Experimental Results

Accuracy rises from 0.76 to 0.87 when the predictor is enabled. Runtime drops by 31 percent relative to quicksort on nearly sorted inputs. An ablation shows that removing the encoder hurts accuracy by 4 points.

6 Conclusion

We presented a learned sorter that is both faster and more accurate than the baselines. Future work will extend it to streaming data.

References

[1] D. Knuth. The Art of Computer Programming, Volume 3. Addison-Wesley, 1998.

[2] T. Kraska et al. The case for learned index structures. SIGMOD 2018.